Sanity check – Do LLMs understand the representations of our diagrams and how deep is that understanding?

Introduction and Test 1 - JSON

Aim is to evaluate how feasible is the idea of enhancing the LLM context with the diagram information by discovering if the model can actually understand the representations of selected diagrams.

Our representation is JSON with multiple levels of nesting, containing the individual parts of the state diagram (state nodes, transition edges etc.).

We will conduct this experiment by manually copy-pasting the actual serialized diagram representation (obtained from our implementation) into the LLM models of 2 popular AI tools, that being Anthropic's Claude and OpenAI's ChatGPT. This is synonymous with how the actual VS Code AI integration of our implementation works.

Used prompts:

1.	I am testing LLMs understanding of my diagram json representation.
2.	so can you explain this diagram? All the nodes and the transitions? [followed by representation for Scenario 1]
3.	Ok lets maybe try another one, this one is unforgiving with its complexity. To make it more concise you can focus only on handleItems of mode state, maybe add some summary at the end containing how many state variables and how many mutators are there... [followed by representation for Scenario 2]

In chat conversation order with new conversation created for each Test type.

1. prompt is overall brief and introductory, vague on purpose no extra information about the data structure. We are testing worst case scenario where models would receive minimal to no context, exact prompt wording does not matter as long as it is not directly deceptive or too concrete…
In real life (our VS Code extension implementation), models will obtain the appropriate instructions and description of the representation as needed.

2. / scenario 1, prompt contains the first task for simple to moderately complex ifElseChain.tsx sample.

3. / scenario 2, is the same task for more complex switchNLoops.tsx with additional instruction to test the model ability to filter or process the data selectively.

Scenario 1 - JSON for ifElseChain.tsx

{"component":{"name":"IfElseChainComplex","pos":{"line":11,"col":1},"exportName":"default","declarationKind":"function"},"stateVariables":[{"id":"3385043882","hook":"useState","name":"phase","setterName":"setPhase","initializerText":"\"idle\"","typeText":"\"idle\" | \"checking\" | \"small\" | \"medium\" | \"large\" | \"done\"","pos":{"line":12,"col":10},"states":[{"id":"745384503","nodeType":"state-update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":25,"col":5},"label":"\"checking\""},{"id":"1400225767","nodeType":"state-update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":33,"col":7},"label":"\"idle\""},{"id":"4059114306","nodeType":"state-update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":46,"col":7},"l

```
abel":"\"small\""},{"id":"2915095121","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":48,"col":7},"l
abel":"\"medium\""},{"id":"199049892","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":50,"col":7},"l
abel":"\"large\""},{"id":"700624121","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":54,"col":5},"l
abel":"\"done\""}],"mutators":[{"id":"96452645","name":"doInit","pos":{"line":20,"col":18},"type":"arrow-
function","nodes":[{"id":"745384503","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":25,"col":5},"l
abel":"\"checking\""},{"id":"1427359308","nodeType":"control-
flow","kind":"entry","label":"doInit","pos":{"line":20,"col":24}},{"id":"502171564","nodeType":"control-
flow","kind":"exit","label":"exit
\"idk\"","pos":{"line":26,"col":5}}],"transitions":[{"id":"1194467231","fromNodeId":"1427359308","toNodeId
":"745384503","kind":"normal"},{"id":"1339748348","fromNodeId":"745384503","toNodeId":"502171564","
kind":"normal"}]},{"id":"1620845592","name":"innrCheck","pos":{"line":32,"col":23},"type":"arrow-
function","nodes":[{"id":"1400225767","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":33,"col":7},"l
abel":"\"idle\""},{"id":"4005628753","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":37,"col":9},"l
abel":"\"checking\""},{"id":"2461057572","nodeType":"control-
flow","kind":"entry","label":"innrCheck","pos":{"line":32,"col":29}},{"id":"2111933513","nodeType":"control-
flow","kind":"decision","label":"value","pos":{"line":35,"col":7}},{"id":"2193564554","nodeType":"control-
flow","kind":"merge","pos":{"line":35,"col":7}},{"id":"4043707519","nodeType":"control-
flow","kind":"exit","pos":{"line":32,"col":29}}],"transitions":[{"id":"1601484938","fromNodeId":"2461057572
","toNodeId":"1400225767","kind":"normal"},{"id":"2056374782","fromNodeId":"1400225767","toNodeId":
"2111933513","kind":"normal"},{"id":"3364260272","fromNodeId":"2111933513","toNodeId":"4005628753
","kind":"then","label":"[true]"},{"id":"1502530256","fromNodeId":"4005628753","toNodeId":"2193564554"
,"kind":"normal"},{"id":"1882240166","fromNodeId":"2111933513","toNodeId":"2193564554","kind":"else",
"label":"[false]"},{"id":"2870426837","fromNodeId":"2193564554","toNodeId":"4043707519","kind":"norma
l"}]},{"id":"3471131163","name":"handleProcess","pos":{"line":29,"col":3},"type":"function","args":"value:
number","nodes":[{"id":"3752981291","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":44,"col":7},"l
abel":"\"idle\""},{"id":"4059114306","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":46,"col":7},"l
abel":"\"small\""},{"id":"2915095121","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":48,"col":7},"l
abel":"\"medium\""},{"id":"199049892","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":50,"col":7},"l
abel":"\"large\""},{"id":"700624121","nodeType":"state-
update","stateVariableId":"3385043882","setterName":"setPhase","kind":"direct","pos":{"line":54,"col":5},"l
abel":"\"done\""},{"id":"623438991","nodeType":"control-
flow","kind":"entry","label":"handleProcess","pos":{"line":29,"col":41}},{"id":"131665883","nodeType":"cont
rol-flow","kind":"decision","label":"value <
0","pos":{"line":43,"col":5}},{"id":"322985586","nodeType":"control-flow","kind":"decision","label":"value
=== 0","pos":{"line":45,"col":12}}vagu,{"id":"188542464","nodeType":"control-
flow","kind":"decision","label":"value <
10","pos":{"line":47,"col":12}},{"id":"2631629035","nodeType":"control-
flow","kind":"merge","pos":{"line":47,"col":12}},{"id":"850776065","nodeType":"control-
flow","kind":"merge","pos":{"line":45,"col":12}},{"id":"346969252","nodeType":"control-
flow","kind":"merge","pos":{"line":43,"col":5}},{"id":"2793433764","nodeType":"control-
flow","kind":"exit","pos":{"line":29,"col":41}}],"transitions":[{"id":"1655864250","fromNodeId":"623438991"
,"toNodeId":"131665883","kind":"normal"},{"id":"3270600210","fromNodeId":"131665883","toNodeId":"37
52981291","kind":"then","label":"[true]"},{"id":"3853327165","fromNodeId":"131665883","toNodeId":"322
985586","kind":"else","label":"[false]"},{"id":"318636969","fromNodeId":"322985586","toNodeId":"4059114
```

306","kind":"then","label":"[true]"},{"id":"3777208555","fromNodeId":"322985586","toNodeId":"18854246
4","kind":"else","label":"[false]"},{"id":"3119220575","fromNodeId":"188542464","toNodeId":"2915095121"
,"kind":"then","label":"[true]"},{"id":"3000669350","fromNodeId":"188542464","toNodeId":"199049892","ki
nd":"else","label":"[false]"},{"id":"273737806","fromNodeId":"2915095121","toNodeId":"2631629035","kin
d":"normal"},{"id":"144849192","fromNodeId":"199049892","toNodeId":"2631629035","kind":"normal"},{"i
d":"3736544152","fromNodeId":"4059114306","toNodeId":"850776065","kind":"normal"},{"id":"416225297
","fromNodeId":"2631629035","toNodeId":"850776065","kind":"normal"},{"id":"903667531","fromNodeId":
"3752981291","toNodeId":"346969252","kind":"normal"},{"id":"3771227525","fromNodeId":"850776065","
toNodeId":"346969252","kind":"normal"},{"id":"3894967882","fromNodeId":"346969252","toNodeId":"700
624121","kind":"normal"},{"id":"1076679036","fromNodeId":"700624121","toNodeId":"2793433764","kind"
:"normal"}]}]},{"id":"2352782266","hook":"useState","name":"count","setterName":"setCount","typeText":"
number","pos":{"line":13,"col":10},"states":[{"id":"3288247318","nodeType":"state-
update","stateVariableId":"2352782266","setterName":"setCount","kind":"direct","pos":{"line":16,"col":5},"l
abel":"0"},{"id":"925429536","nodeType":"state-
update","stateVariableId":"2352782266","setterName":"setCount","kind":"expression","pos":{"line":30,"col"
:5},"label":"value"},{"id":"2293718465","nodeType":"state-
update","stateVariableId":"2352782266","setterName":"setCount","kind":"expression","pos":{"line":60,"col"
:30},"label":"count! + 10"}],"mutators":[{"id":"2543190788","name":"<render-
body>","pos":{"line":11,"col":1},"type":"function","nodes":[{"id":"3288247318","nodeType":"state-
update","stateVariableId":"2352782266","setterName":"setCount","kind":"direct","pos":{"line":16,"col":5},"l
abel":"0"},{"id":"2204760367","nodeType":"control-flow","kind":"entry","label":"<render-
body>","pos":{"line":11,"col":46}},{"id":"4143953516","nodeType":"control-
flow","kind":"decision","label":"count ==
undefined","pos":{"line":15,"col":3}},{"id":"1869423133","nodeType":"control-
flow","kind":"merge","pos":{"line":15,"col":3}},{"id":"2748615447","nodeType":"control-
flow","kind":"exit","label":"return <> <button onClick={() => handleProcess(count! + 1)}>Run</button>
<button
onCli...","pos":{"line":58,"col":3}}],"transitions":[{"id":"3836737277","fromNodeId":"2204760367","toNodeI
d":"4143953516","kind":"normal"},{"id":"2276903582","fromNodeId":"4143953516","toNodeId":"32882473
18","kind":"then","label":"[true]"},{"id":"2591913946","fromNodeId":"3288247318","toNodeId":"18694231
33","kind":"normal"},{"id":"3729246923","fromNodeId":"4143953516","toNodeId":"1869423133","kind":"el
se","label":"[false]"},{"id":"39825771","fromNodeId":"1869423133","toNodeId":"2748615447","kind":"norm
al"}]},{"id":"3360255919","name":"handleProcess","pos":{"line":29,"col":3},"type":"function","args":"value:
number","nodes":[{"id":"925429536","nodeType":"state-
update","stateVariableId":"2352782266","setterName":"setCount","kind":"expression","pos":{"line":30,"col"
:5},"label":"value"},{"id":"3562600700","nodeType":"control-
flow","kind":"entry","label":"handleProcess","pos":{"line":29,"col":41}},{"id":"3015197983","nodeType":"co
ntrol-
flow","kind":"exit","pos":{"line":29,"col":41}}],"transitions":[{"id":"1165347568","fromNodeId":"3562600700
","toNodeId":"925429536","kind":"normal"},{"id":"527437630","fromNodeId":"925429536","toNodeId":"30
15197983","kind":"normal"}]},{"id":"1867133108","name":"onClick","pos":{"line":60,"col":22},"type":"arrow
-function","nodes":[{"id":"2293718465","nodeType":"state-
update","stateVariableId":"2352782266","setterName":"setCount","kind":"expression","pos":{"line":60,"col"
:30},"label":"count! + 10"},{"id":"3736109947","nodeType":"control-
flow","kind":"entry","label":"onClick","pos":{"line":60,"col":28}},{"id":"1173259662","nodeType":"control-
flow","kind":"exit","pos":{"line":60,"col":28}}],"transitions":[{"id":"3135446956","fromNodeId":"3736109947
","toNodeId":"2293718465","kind":"normal"},{"id":"687597660","fromNodeId":"2293718465","toNodeId":"
1173259662","kind":"normal"}]}]}],"source":"samples\\ifElseChain.tsx"}

Estimated AI token count: *3009 (estimated by https://llm-calculator.com/; GPT-4o)*

Scenario 2 - JSON for switchNLoops.tsx

{"component":{"name":"SwitchWithLoops","pos":{"line":3,"col":1},"exportName":"default","declarationKind":"function"},"stateVariables":[{"id":"3816675366","hook":"useState","name":"mode","setterName":"setMode","initializerText":"\"idle\"","typeText":"\"idle\" | \"scan\" | \"match\" | \"skip\" | \"error\" | \"complete\"","pos":{"line":4,"col":10},"states":[{"id":"2367670358","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":13,"col":7},"label":"\"idle\""},{"id":"88022199","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":18,"col":5},"label":"\"scan\""},{"id":"3885211920","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":34,"col":9},"label":"\"match\""},{"id":"1535183815","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":38,"col":9},"label":"\"skip\""},{"id":"1404204274","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":42,"col":9},"label":"\"error\""},{"id":"3674119774","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":95,"col":3},"label":"\"complete\""}],"mutators":[{"id":"177170579","name":"func12","pos":{"line":9,"col":30},"type":"arrow-function","args":"idk: string","nodes":[{"id":"2367670358","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":13,"col":7},"label":"\"idle\""},{"id":"3682333420","nodeType":"control-flow","kind":"entry","label":"func12","pos":{"line":9,"col":47}},{"id":"2483458591","nodeType":"control-flow","kind":"decision","label":"mode == \"complete\"","pos":{"line":12,"col":5}},{"id":"269498684","nodeType":"control-flow","kind":"merge","pos":{"line":12,"col":5}},{"id":"2662296193","nodeType":"control-flow","kind":"exit","pos":{"line":9,"col":47}}],"transitions":[{"id":"2867251430","fromNodeId":"3682333420","toNodeId":"2483458591","kind":"normal"},{"id":"2937590571","fromNodeId":"2483458591","toNodeId":"2367670358","kind":"then","label":"[true]"},{"id":"93252150","fromNodeId":"2367670358","toNodeId":"269498684","kind":"normal"},{"id":"2691665213","fromNodeId":"2483458591","toNodeId":"269498684","kind":"else","label":"[false]"},{"id":"3454802310","fromNodeId":"269498684","toNodeId":"2662296193","kind":"normal"}]},{"id":"3739641387","name":"processItems","pos":{"line":17,"col":3},"type":"function","args":"type: string, items?: number[]","nodes":[{"id":"88022199","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":18,"col":5},"label":"\"scan\""},{"id":"3885211920","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":34,"col":9},"label":"\"match\""},{"id":"1535183815","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":38,"col":9},"label":"\"skip\""},{"id":"1404204274","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":42,"col":9},"label":"\"error\""},{"id":"1758339376","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":55,"col":9},"label":"\"idle\""},{"id":"2881501837","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":67,"col":11},"label":"\"error\""},{"id":"205779507","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":72,"col":11},"label":"\"skip\""},{"id":"70652892","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":77,"col":11},"label":"\"match\""},{"id":"3674119774","nodeType":"state-update","stateVariableId":"3816675366","setterName":"setMode","kind":"direct","pos":{"line":95,"col":3},"label":"\"complete\""},{"id":"3721046495","nodeType":"control-flow","kind":"entry","label":"processItems","pos":{"line":17,"col":57}},{"id":"3578961293","nodeType":"control-flow","kind":"decision","label":"!items","pos":{"line":21,"col":5}},{"id":"1072029985","nodeType":"control-flow","kind":"exit","pos":{"line":23,"col":7}},{"id":"3207890600","nodeType":"control-flow","kind":"switch-

```
decision","label":"type","pos":{"line":29,"col":5}},{"id":"1527332951","nodeType":"control-
flow","kind":"exit","pos":{"line":39,"col":9}},{"id":"2155460848","nodeType":"control-
flow","kind":"merge","pos":{"line":29,"col":5}},{"id":"1116015517","nodeType":"control-flow","kind":"loop-
decision","label":"count-- > 0","pos":{"line":59,"col":5}},{"id":"2206506855","nodeType":"control-
flow","kind":"loop-decision","label":"i <
items.length","pos":{"line":60,"col":7}},{"id":"1373976025","nodeType":"control-
flow","kind":"decision","label":"items[i] <
0","pos":{"line":66,"col":9}},{"id":"2307902917","nodeType":"control-
flow","kind":"decision","label":"items[i] ===
0","pos":{"line":71,"col":9}},{"id":"526769459","nodeType":"control-flow","kind":"decision","label":"items[i]
% 2 === 0","pos":{"line":76,"col":9}},{"id":"2291310596","nodeType":"control-
flow","kind":"merge","pos":{"line":76,"col":9}},{"id":"523023049","nodeType":"control-
flow","kind":"merge","pos":{"line":60,"col":7}},{"id":"770977296","nodeType":"control-
flow","kind":"exit","pos":{"line":17,"col":57}}],"transitions":[{"id":"3815402159","fromNodeId":"3721046495
","toNodeId":"88022199","kind":"normal"},{"id":"1132313439","fromNodeId":"88022199","toNodeId":"357
8961293","kind":"normal"},{"id":"1460829605","fromNodeId":"3578961293","toNodeId":"1072029985","kin
d":"then","label":"[true]"},{"id":"1951966814","fromNodeId":"3578961293","toNodeId":"3207890600","kin
d":"else","label":"[false]"},{"id":"3946800658","fromNodeId":"3207890600","toNodeId":"3885211920","kind
":"case","label":"[case] \"something\" | \"others\" | \"fast\"","rawConditionText":"\"something\" |
\"others\" |
\"fast\""},{"id":"3973304587","fromNodeId":"3207890600","toNodeId":"1535183815","kind":"case","label":
"[case]
\"slow\"","rawConditionText":"\"slow\""},{"id":"1974416068","fromNodeId":"1535183815","toNodeId":"152
7332951","kind":"normal"},{"id":"3398773617","fromNodeId":"3207890600","toNodeId":"1404204274","kin
d":"case","label":"[case]
\"broken\"","rawConditionText":"\"broken\""},{"id":"402576008","fromNodeId":"3207890600","toNodeId":"
1758339376","kind":"default","label":"[default]"},{"id":"2083637201","fromNodeId":"3885211920","toNode
Id":"2155460848","kind":"normal"},{"id":"3492713905","fromNodeId":"1404204274","toNodeId":"2155460
848","kind":"normal"},{"id":"4087036334","fromNodeId":"3207890600","toNodeId":"2155460848","kind":"c
ase","label":"[case]
\"error\"","rawConditionText":"\"error\""},{"id":"4054138506","fromNodeId":"1758339376","toNodeId":"21
55460848","kind":"normal"},{"id":"3994488598","fromNodeId":"2155460848","toNodeId":"1116015517","ki
nd":"normal"},{"id":"2012486596","fromNodeId":"1116015517","toNodeId":"2206506855","kind":"then","la
bel":"[true]"},{"id":"2880447550","fromNodeId":"2206506855","toNodeId":"1373976025","kind":"then","la
bel":"[true]"},{"id":"2434479504","fromNodeId":"1373976025","toNodeId":"2881501837","kind":"then","la
bel":"[true]"},{"id":"2051762350","fromNodeId":"1373976025","toNodeId":"2307902917","kind":"else","lab
el":"[false]"},{"id":"1053672919","fromNodeId":"2307902917","toNodeId":"205779507","kind":"then","label
":"[true]"},{"id":"4272124239","fromNodeId":"2307902917","toNodeId":"526769459","kind":"else","label":"
[false]"},{"id":"3112748071","fromNodeId":"526769459","toNodeId":"70652892","kind":"then","label":"[tru
e]"},{"id":"3092221956","fromNodeId":"70652892","toNodeId":"2291310596","kind":"normal"},{"id":"1227
115536","fromNodeId":"526769459","toNodeId":"2291310596","kind":"else","label":"[false]"},{"id":"278932
6459","fromNodeId":"2291310596","toNodeId":"2206506855","kind":"loop","label":"[loop]"},{"id":"234706
5884","fromNodeId":"205779507","toNodeId":"2206506855","kind":"loop","label":"[loop]"},{"id":"1117584
587","fromNodeId":"2206506855","toNodeId":"523023049","kind":"else","label":"[false]"},{"id":"167209102
6","fromNodeId":"2881501837","toNodeId":"523023049","kind":"normal"},{"id":"191752876","fromNodeId
":"523023049","toNodeId":"1116015517","kind":"loop","label":"[loop]"},{"id":"3489507015","fromNodeId":
"1116015517","toNodeId":"3674119774","kind":"else","label":"[false]"},{"id":"2639722415","fromNodeId":"
3674119774","toNodeId":"770977296","kind":"normal"}]}]},{"id":"925063818","hook":"useState","name":"i
ndex","setterName":"setIndex","initializerText":"0","typeText":"number","pos":{"line":5,"col":10},"states":[{"
id":"740410588","nodeType":"state-
update","stateVariableId":"925063818","setterName":"setIndex","kind":"direct","pos":{"line":7,"col":3},"lab
el":"1"},{"id":"2031999384","nodeType":"state-
update","stateVariableId":"925063818","setterName":"setIndex","kind":"direct","pos":{"line":19,"col":5},"la
bel":"0"},{"id":"2081754794","nodeType":"state-
```

update","stateVariableId":"925063818","setterName":"setIndex","kind":"expression","pos":{"line":64,"col":9},"label":"i"}],"mutators":[{"id":"490143892","name":"<render-body>","pos":{"line":3,"col":1},"type":"function","nodes":[{"id":"740410588","nodeType":"state-update","stateVariableId":"925063818","setterName":"setIndex","kind":"direct","pos":{"line":7,"col":3},"label":"1"},{"id":"3181532511","nodeType":"control-flow","kind":"entry","label":"<render-body>","pos":{"line":3,"col":43}},{"id":"2224702489","nodeType":"control-flow","kind":"exit","label":"return <> <button onClick={() => processItems(\"fast\", [1, 2, 0, 4])}>Process</button> ...","pos":{"line":100,"col":3}}],"transitions":[{"id":"1836757544","fromNodeId":"3181532511","toNodeId":"740410588","kind":"normal"},{"id":"558942648","fromNodeId":"740410588","toNodeId":"2224702489","kind":"normal"}]},{"id":"1231352507","name":"processItems","pos":{"line":17,"col":3},"type":"function","args":"type: string, items?: number[]","nodes":[{"id":"2031999384","nodeType":"state-update","stateVariableId":"925063818","setterName":"setIndex","kind":"direct","pos":{"line":19,"col":5},"label":"0"},{"id":"2081754794","nodeType":"state-update","stateVariableId":"925063818","setterName":"setIndex","kind":"expression","pos":{"line":64,"col":9},"label":"i"},{"id":"550693652","nodeType":"control-flow","kind":"entry","label":"processItems","pos":{"line":17,"col":57}},{"id":"317770123","nodeType":"control-flow","kind":"decision","label":"!items","pos":{"line":21,"col":5}},{"id":"3309593840","nodeType":"control-flow","kind":"exit","pos":{"line":23,"col":7}},{"id":"4235766631","nodeType":"control-flow","kind":"switch-decision","label":"type","pos":{"line":29,"col":5}},{"id":"1065567023","nodeType":"control-flow","kind":"exit","pos":{"line":39,"col":9}},{"id":"3568545102","nodeType":"control-flow","kind":"loop-decision","label":"count-- > 0","pos":{"line":59,"col":5}},{"id":"53431192","nodeType":"control-flow","kind":"loop-decision","label":"i < items.length","pos":{"line":60,"col":7}},{"id":"2000687429","nodeType":"control-flow","kind":"exit","pos":{"line":17,"col":57}}],"transitions":[{"id":"2718907274","fromNodeId":"550693652","toNodeId":"2031999384","kind":"normal"},{"id":"4259765101","fromNodeId":"2031999384","toNodeId":"317770123","kind":"normal"},{"id":"1673332866","fromNodeId":"317770123","toNodeId":"3309593840","kind":"then","label":"[true]"},{"id":"3350880475","fromNodeId":"317770123","toNodeId":"4235766631","kind":"else","label":"[false]"},{"id":"3290646535","fromNodeId":"4235766631","toNodeId":"1065567023","kind":"case","label":"[case] \"slow\"","rawConditionText":"\"slow\""},{"id":"499878410","fromNodeId":"4235766631","toNodeId":"3568545102","kind":"default","label":"[default]"},{"id":"1647230596","fromNodeId":"3568545102","toNodeId":"53431192","kind":"then","label":"[true]"},{"id":"1185415729","fromNodeId":"53431192","toNodeId":"2081754794","kind":"then","label":"[true]"},{"id":"2356403115","fromNodeId":"2081754794","toNodeId":"53431192","kind":"loop","label":"[loop]"},{"id":"383961008","fromNodeId":"53431192","toNodeId":"3568545102","kind":"loop","label":"[loop]"},{"id":"2348354312","fromNodeId":"3568545102","toNodeId":"2000687429","kind":"else","label":"[false]"}]}]}],"source":"samples\\switchNLoops.tsx"}

Estimated AI token count: *3920 (estimated by https://llm-calculator.com/ ; GPT-4o)*

Both models received exactly the same prompt and data, with occasional steering instruction we will describe later when present. The JSON representation is without any modifications specific for this test.

Respective source code for all mentioned scenarios can be found in the testing dataset (under samples).

Evaluation criteria

Responses for each model will be examined for each scenario and manually evaluated with following metrics:

Metric Scenario	State variables identified (SV)	Mutator functions identified (MF)	Unique states identified (US)	Logic analyses score (LAS)
Scenario 1	u/stateVarCnt	m/muCnt	us/uniqSttCnt	ls/10
Scenario N	u/stateVarCnt	m/muCnt	us/uniqSttCnt	ls/10

- **State variables identified (SV), Mutator functions identified (MF)** and **Unique states identified (US)** – Measure how much of a specific metric (u, m, us, ls) has been successfully identified by the model, out of total count of respective metric which are present in the data (stateVarCnt, muCnt, uniqSttCnt).
    - Any mention of the metric (description, bullet point with the given state variable/state/mutator etc.) in the models response will receive **1 point.**
    - If the given metric is mentioned, but misinterpreted or meaningfully different, model receives **0.5 point**. If not mentioned at all **0 points**.
- **Logic analyses score (LAS)** – Measure how correct was the models understanding of the state diagrams themselves, the underlying logic, the control flow and overall semantics.
  Model can receive max 10 points where:
    - For non critical error or minor misinterpretation (state something that is not entirely correct, misreading something etc.) in details, -**0.5** or **-1** point is subtracted based on severity.
    - For major semantic mistakes like failing to mention core part of the logic, reinventing nonexciting parts of the flow, or incorrectly changing semantics in a meaningful way (changing transitions, order of operations, omitting important details etc.) will result in **-2 points** subtracted.

For Scenario 2 where selection is tested, we have locked the **State variables identified, Mutator functions identified** counts to max 1. If the model correctly selects and focuses on the requested state variable and mutator he receives **1 point.** If he identifies the required but also proceeds to overly focus on other unwanted ones, he receives **0.5 point** and **0** in case of not explaining it at all (or explaining completely different mutator or state variable).
Unique states identified are also specific to that selected mutator, not the whole component.
Model can't receive negative score, 0 is the lowest for every metric. And higher = better for every metric.

Calculating the overall accuracy is a result of following weighted avg.:
(u/stateVarCnt)*0.1 + (m/muCnt)*0.1 + (us/uniqSttCnt)*0.25 + (ls/10)*0.55 =
SV*0.1 + MF*0.1 + US*0.25 + LAS*0.55, prioritizing the states and logic of the diagram.

Rational behind subtracting the points will be adequately documented below the tables.

ChatGPT 5.3 (subscription)

Response for Scenario 1:

I'll read it as two separate state-variable diagrams: one for `phase`, one for `count`. The important part is that every mutator has local control-flow nodes plus state-update nodes, and transitions describe execution order.

Thought for a few seconds

Yes. This diagram represents one React component:

```
function IfElseChainComplex()
```

It found **two state variables**:

```
const [phase, setPhase] = useState("idle")
const [count, setCount] = useState<number>()
```

---

1. `phase` state diagram

`phase` can become:

```
"idle" | "checking" | "small" | "medium" | "large" | "done"
```

Mutator: `doInit`

Nodes:

```
entry: doInit
update: setPhase("checking")
exit: return "idk"
```

Transitions:

```
doInit entry
  -> setPhase("checking")
  -> return "idk"
```

Meaning:

```
const doInit = () => {
  setPhase("checking");
  return "idk";
}
```

This one is simple and linear.

Mutator: `innrCheck`

Nodes:

```
entry: innrCheck
update: setPhase("idle")
decision: value
update: setPhase("checking")
exit
```

Transitions:

```
entry
  -> setPhase("idle")
  -> decision: value

if value is true:
  -> setPhase("checking")
  -> exit

if value is false:
  -> exit
```

Meaning:

```
const innrCheck = () => {
  setPhase("idle");

  if (value) {
    setPhase("checking");
  }
}
```

So `setPhase("idle")` always happens.
`setPhase("checking")` only happens when `value` is truthy.

Mutator: `handleProcess(value: number)`

This is the largest one.

Nodes:

```
entry: handleProcess

decision: value < 0
update: setPhase("idle")

decision: value === 0
update: setPhase("small")

decision: value < 10
update: setPhase("medium")
update: setPhase("large")

merge after value < 10
merge after value === 0
merge after value < 0

update: setPhase("done")
```

```
exit
```

Transitions:

```
entry
  -> decision: value < 0
```

First condition:

```
if value < 0:
  -> setPhase("idle")
  -> merge after value < 0
```

Else branch:

```
if value < 0 is false:
  -> decision: value === 0
```

Second condition:

```
if value === 0:
  -> setPhase("small")
  -> merge after value === 0
```

Else branch:

```
if value === 0 is false:
  -> decision: value < 10
```

Third condition:

```
if value < 10:
  -> setPhase("medium")
  -> merge after value < 10
```

Final else:

```
if value < 10 is false:
  -> setPhase("large")
  -> merge after value < 10
```

Then all branches join:

```
merge after value < 10
  -> merge after value === 0
  -> merge after value < 0
  -> setPhase("done")
  -> exit
```

Equivalent source shape:

```
function handleProcess(value: number) {
  if (value < 0) {
    setPhase("idle");
  } else if (value === 0) {
    setPhase("small");
  } else if (value < 10) {
```

```
    setPhase("medium");
  } else {
    setPhase("large");
  }

  setPhase("done");
}
```

Important semantic detail: regardless of which branch runs, `setPhase("done")` happens afterward. So the final `phase` value after `handleProcess` is probably always `"done"`.

---

2. `count` state diagram

`count` is a `number`.

It has three updates:

```
setCount(0)
setCount(value)
setCount(count! + 10)
```

Mutator: `<render-body>`

This is code that runs directly during render.

Nodes:

```
entry: <render-body>
decision: count == undefined
update: setCount(0)
exit: return JSX
```

Transitions:

```
entry
  -> decision: count == undefined

if true:
  -> setCount(0)
  -> return JSX

if false:
  -> return JSX
```

Equivalent source shape:

```
if (count == undefined) {
  setCount(0);
}

return (
  <>
    ...
  </>
```

```
);
```

This is worth flagging: calling `setCount` during render is usually suspicious in React, even if guarded.

Mutator: `handleProcess(value: number)`

Nodes:

```
entry: handleProcess
update: setCount(value)
exit
```

Transitions:

```
entry
  -> setCount(value)
  -> exit
```

Equivalent:

```
function handleProcess(value: number) {
  setCount(value);
}
```

This is the `count` part of the same `handleProcess` function. Your diagram splits it by state variable, so `handleProcess` appears once under `phase` and once under `count`.

Mutator: `onClick`

Nodes:

```
entry: onClick
update: setCount(count! + 10)
exit
```

Transitions:

```
entry
  -> setCount(count! + 10)
  -> exit
```

Equivalent:

```
onClick={() => setCount(count! + 10)}
```

Overall meaning

The diagram is not one single global control-flow graph. It is grouped by **state variable**.

So:

```
phase
  doInit
  innrCheck
  handleProcess

count
  <render-body>
  handleProcess
  onClick
```

Each mutator contains:

```
entry nodes       = function/event start
decision nodes    = if / condition checks
state-update nodes = setter calls
merge nodes       = branches joining back together
exit nodes        = function end / return
transitions       = execution paths between nodes
```

The diagram looks structurally valid and understandable. The only slightly confusing part is that the same real function, `handleProcess`, is split into two separate diagrams: one for `phase` updates and one for `count` updates. That is fine if your UI is state-variable-centric, but if the user asks "what does handleProcess do?", the LLM needs to mentally combine both parts:

```
function handleProcess(value: number) {
  setCount(value);

  if (value < 0) setPhase("idle");
  else if (value === 0) setPhase("small");
  else if (value < 10) setPhase("medium");
  else setPhase("large");

  setPhase("done");
}
```

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
2/2	6/6	9/9	10/10

Response for Scenario 2:

Yes — this one is more complex, but still understandable.

The component has **2 state variables**:

```
mode
index
```

Total mutators:

```
mode: 2 mutators
index: 2 mutators
total: 4 mutators
```

Focusing only on `mode.processItems`.

`mode.processItems(type, items?)`

This mutator updates:

`setMode(...)`
Possible updates inside it: Yes — this one is much more complex, but still understandable.

Small naming note: I don't see `handleItems`; the relevant mutator is:

```
processItems(type: string, items?: number[])
```

for the `mode` state.

`mode` state

```
mode: "idle" | "scan" | "match" | "skip" | "error" | "complete"
```

`processItems` can set:

```
scan
match
skip
error
idle
complete
```

`mode.processItems(...)`

High-level behavior

The flow is basically:

```
function processItems(type: string, items?: number[]) {
  setMode("scan");

  if (!items) return;

  switch (type) {
    case "something":
    case "others":
    case "fast":
      setMode("match");
      break;

    case "slow":
      setMode("skip");
      return;

    case "broken":
      setMode("error");
      break;

    case "error":
      break;

    default:
      setMode("idle");
  }

  while (count-- > 0) {
    for (...) {
      if (items[i] < 0) {
        setMode("error");
        break / exit inner loop-ish;
      } else if (items[i] === 0) {
        setMode("skip");
        continue;
      } else if (items[i] % 2 === 0) {
        setMode("match");
      }
    }
  }

  setMode("complete");
}
```

Exact loop semantics depend on the original source, but that is the structure implied by the graph.

Nodes

Entry

```
3721046495: entry processItems
```

Start of the function.

First transition:

```
entry -> setMode("scan")
```

So `scan` always happens first.

Initial update

```
88022199: setMode("scan")
```

Then it checks:

```
3578961293: decision !items
```

Transitions:

```
if !items true:
  -> exit at line 23

if !items false:
  -> switch(type)
```

So if `items` is missing, the function stops early after setting mode to `"scan"`.

Switch section

Switch decision

```
3207890600: switch-decision type
```

This branches based on `type`.

Case `"something" | "others" | "fast"`

```
type in ["something", "others", "fast"]
  -> setMode("match")
  -> switch merge
```

Node:

```
3885211920: setMode("match")
```

This case continues after the switch.

Case `"slow"`

```
type === "slow"
  -> setMode("skip")
  -> exit
```

Nodes:

```
1535183815: setMode("skip")
1527332951: exit
```

This is an early exit. It does **not** reach the loops or final `"complete"` update.

Case `"broken"`

```
type === "broken"
  -> setMode("error")
  -> switch merge
```

Node:

```
1404204274: setMode("error")
```

Then execution continues into the loop section.

Case `"error"`

```
type === "error"
  -> switch merge
```

No `setMode(...)` in that case. It just falls through to the code after the switch.

Default case

```
default
  -> setMode("idle")
  -> switch merge
```

Node:

```
1758339376: setMode("idle")
```

Then execution continues.

After switch

All non-exiting switch paths meet here:

```
2155460848: merge
```

Then:

```
merge -> outer loop decision
```

Loop section

There are two loop decisions:

```
1116015517: count-- > 0
2206506855: i < items.length
```

This looks like a nested loop:

```
while (count-- > 0) {
  for (...; i < items.length; ...) {
    ...
  }
}
```

Outer loop: `count-- > 0`

```
if count-- > 0 true:
  -> inner loop condition i < items.length

if false:
  -> setMode("complete")
  -> exit
```

So `complete` only happens after the outer loop finishes normally.

Inner loop: `i < items.length`

```
if i < items.length true:
  -> inspect items[i]

if false:
  -> inner-loop merge
  -> go back to outer loop
```

Node:

```
523023049: merge after inner loop
```

Item checks inside the inner loop

First check: `items[i] < 0`

```
1373976025: decision items[i] < 0
```

If true:

```
-> setMode("error")
-> inner-loop merge
-> outer loop again
```

Node:

```
2881501837: setMode("error")
```

So a negative item sets mode to `"error"` and exits/ends the current inner-loop path.

If false:

```
-> check items[i] === 0
```

Second check: `items[i] === 0`

```
2307902917: decision items[i] === 0
```

If true:

```
-> setMode("skip")
-> loop back to i < items.length
```

Node:

```
205779507: setMode("skip")
```

This behaves like a `continue`.

If false:

```
-> check items[i] % 2 === 0
```

Third check: `items[i] % 2 === 0`

```
526769459: decision items[i] % 2 === 0
```

If true:

```
-> setMode("match")
-> merge
```

```
-> loop back to i < items.length
```

Node:

```
70652892: setMode("match")
```

If false:

```
-> merge
-> loop back to i < items.length
```

So odd positive non-zero values do not change `mode`.

Final update

When the outer loop finishes:

```
1116015517 count-- > 0
  [false] -> setMode("complete")
```

Node:

```
3674119774: setMode("complete")
```

Then:

```
setMode("complete") -> exit
```

So for all paths that do not early-return, the final mode becomes:

```
"complete"
```

Important: many intermediate modes like `"match"`, `"skip"`, `"error"`, and `"idle"` may be overwritten by `"complete"` at the end.

Most important paths

No items

```
setMode("scan")
if !items
return
```

Final mode likely:

```
"scan"
```

Type is `"slow"`

```
setMode("scan")
items exist
type === "slow"
setMode("skip")
return
```

Final mode likely:

```
"skip"
```

Type is `"fast"` / `"something"` / `"others"`

```
setMode("scan")
setMode("match")
run loops
setMode("complete")
```

Final mode likely:

```
"complete"
```

Type is `"broken"`

```
setMode("scan")
setMode("error")
run loops
setMode("complete")
```

Final mode likely:

```
"complete"
```

Even though it becomes `"error"` first, it may later become `"complete"`.

Item is negative

```
items[i] < 0
setMode("error")
continue/exit inner path
eventually setMode("complete")
```

Unless there is an early exit in the original code, final mode still likely becomes `"complete"`.

Item is zero

```
items[i] === 0
setMode("skip")
continue inner loop
```

Item is positive even

```
items[i] % 2 === 0
setMode("match")
continue inner loop
```

Item is positive odd

```
no mode update
continue inner loop
```

Summary

This component has:

```
State variables: 2
- mode
- index
```

Mutator entries in the diagram:

```
Total mutator entries: 4
- mode.func12
- mode.processItems
- index.<render-body>
- index.processItems
```

Unique source-level mutator/function names:

```
3
- func12
- processItems
- <render-body>
```

For `mode.processItems`, there are:

```
State-update nodes: 9
Control-flow nodes: 13
Transitions: 31
```

The key semantic takeaway:

```
processItems usually starts with mode = "scan",
may temporarily set "match", "skip", "error", or "idle",
```

```
and usually ends with mode = "complete",
except for early-return paths like missing items or type === "slow".
```

Evaluated result:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	6/6	10/10

The level at which the ChatGPT seems to understand our representation is near perfect, without any major factual mistakes, it even appears to be able to reconstruct the original code almost perfectly from the representation.

Claude Sonnet 4.6

Response for Scenario 1:

This is a React component flow diagram. Let me break it down comprehensively.

Component Overview

`IfElseChainComplex` — a default-exported React function component with **2 state variables** and **5 mutator functions**.

State Variables

1. `phase` (string union)

Type: `"idle" | "checking" | "small" | "medium" | "large" | "done"`

2. `count` (number)

Mutators / Functions

1. `<render-body>` — manages `count`

The component's top-level render logic.

```
ENTRY
 ↓
[count == undefined?]
  ├─ [true]  → setCount(0) → EXIT (render JSX)
  └─ [false] → EXIT (render JSX)
```

On first render, if `count` is undefined it resets it to `0`, then renders two buttons.

---

2. `doInit` — manages `phase`

A simple arrow function, no conditions.

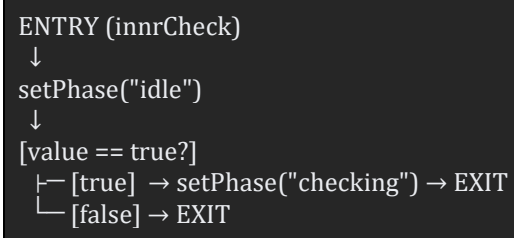
```
ENTRY (doInit)
 ↓
setPhase("checking")
 ↓
EXIT (return "idk")
```

Always sets phase to `"checking"` and returns `"idk"`.

---

3. `innrCheck` — manages `phase`

An arrow function with one conditional branch.

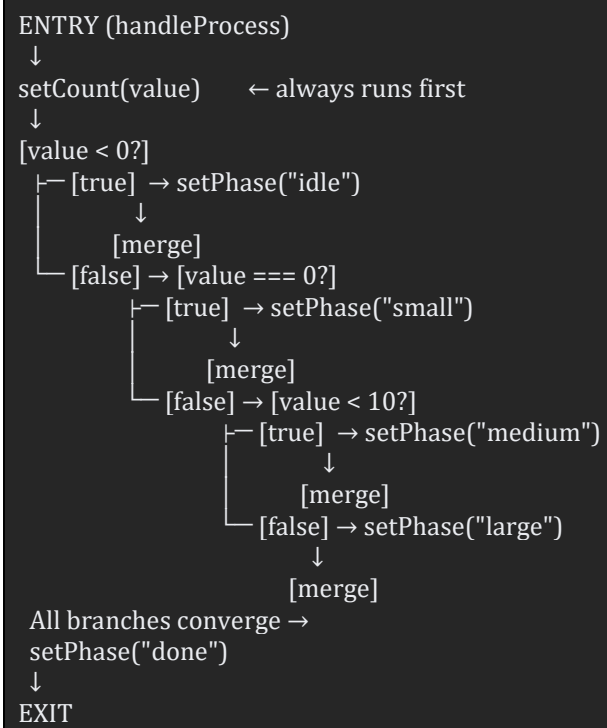


Always sets phase to `"idle"` first, then conditionally sets it back to `"checking"` if `value` is truthy.

---

4. `handleProcess(value: number)` — manages both `phase` and `count`

This is the most complex function — an if/else chain on the numeric `value`.



So the phase logic is:

Condition	Phase set
`value < 0`	`"idle"` → then `"done"`
`value === 0`	`"small"` → then `"done"`
`0 < value < 10`	`"medium"` → then `"done"`
`value >= 10`	`"large"` → then `"done"`

Every path ends with `setPhase("done")` after the if/else chain merges.

---

5. `onClick` (inline arrow on button) — manages `count`

```
ENTRY (onClick)
 ↓
setCount(count! + 10)
 ↓
EXIT
```

A simple inline button handler that increments `count` by 10.

Rendered UI (inferred from exit label)

Two buttons:

- **"Run"** → calls `handleProcess(count! + 1)`
- **Second button** → `onClick` handler that sets `count = count + 10`

Summary

The component models a **multi-phase processing pipeline**: a numeric `count` drives `handleProcess`, which classifies the value into size buckets (`small/medium/large`) and always terminates in `"done"`. Helper functions `doInit` and `innrCheck` manage phase transitions independently.

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
2/2	6/6	9/9	9/10

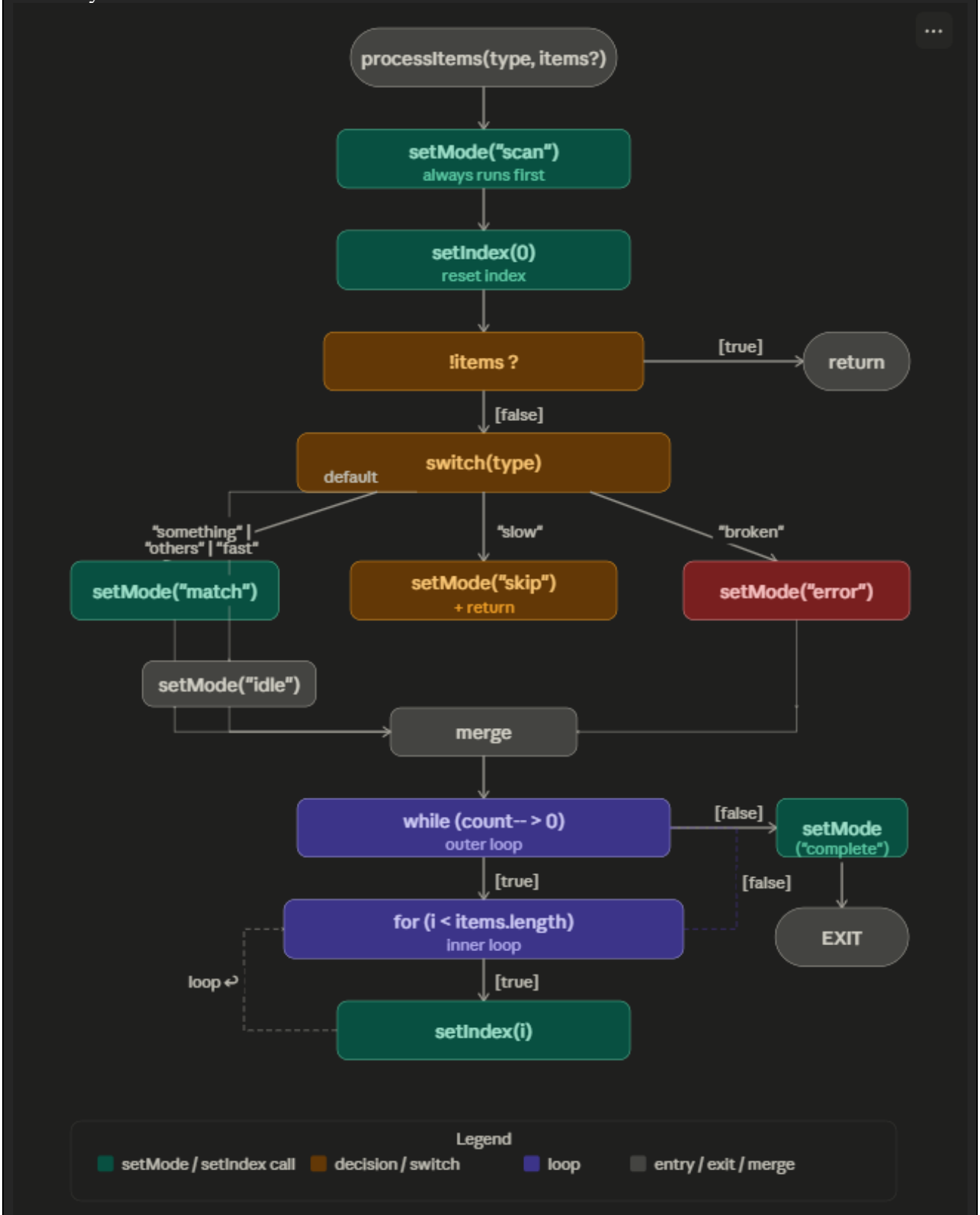
- Claude, like ChatGPT, is mostly on spot with the provided representation.
  However we observed that he did not fully understood that there are separate diagrams for each state variable and mutator and merged handleProcess into one function. His analyses remaining correct, so for this non critical mistake we subtract -1 from LAS.

Claude seems a bit hesitant at providing actual code. So we have decided to nudge him in the right direction by slightly modifying the following prompt:

Ok lets maybe try another one, this one is unforgiving with its complexity. To make it more concise you can focus only on handleItems of mode state. **Try generate a js/ts representation of this mutator.** Maybe add some summary at the end containing how many state variables and how many mutators are there...

Response for Scenario 2:

Here's the TypeScript reconstruction of `handleProcess` as it mutates `mode`, followed by the overall summary.



And here's the TypeScript reconstruction:

```
function processItems(type: string, items?: number[]): void {
```

```
  setMode("scan");   // always first
  setIndex(0);       // reset index

  if (!items) return; // early exit — no items

  // switch on type sets initial phase
  switch (type) {
    case "something":
    case "others":
    case "fast":
      setMode("match");
      break;
    case "slow":
      setMode("skip");
      return;        // early return for "slow"
    case "broken":
      setMode("error");
      break;
    default:
      setMode("idle");
  }

  // outer loop — count is some external/closure variable
  while (count-- > 0) {
    // inner loop — iterate over items
    for (let i = 0; i < items.length; i++) {
      setIndex(i);   // track current position
    }
  }

  setMode("complete"); // always the final state
}
```

A few things worth noting about the logic: the `"slow"` case is the only branch that skips the loops entirely via an early `return`. All other cases (including `default`) fall through to the nested loop structure and terminate with `setMode("complete")`. The `setIndex(i)` inside the inner loop records the current iteration index as React state, which is an unusual pattern — it means every iteration triggers a re-render.

Component summary

	Count
State variables	**3** — `mode`, `index`, `count` (implied by loop; not visible as useState in this JSON)
Mutators on `mode`	**2** — `func12`, `processItems`
Mutators on `index`	**3** — `<render-body>`, `processItems`, `onClick` (inline)
Total distinct mutators	**4** — `<render-body>`, `func12`, `processItems`, inline `onClick`

Based strictly on the JSON, there are **2 explicit useState hooks** (`mode` and `index`). `count` referenced in the outer loop condition appears to be an external or closure variable not declared as a hook in this snapshot.

In this response, Claude has also decided to generate the actual diagram. Although it is a bit rough and missing some parts, but still quite representative.
Key observation is again that the Claude does not seem to completely understand that there are separate sub-diagrams for each mutator function (each in its swimlane) with the relation to the respective state variable, so he focused to much on forcefully merging the 2 separate *processItems* diagrams one for the *phase* and another for *index.*

This is might be expected since the context both models obtained was basically nonexistent…

We have further decided to clarify this detail:

> One slight clarification. The model represents mutators for each state as a separate entity so index::processItems and mode::processItems are modeled separatly so the mode::processItems should not contain index related states

Response after clarification:

Good catch! You're right — each state variable's view of `processItems` only contains the nodes and transitions relevant to *that* state. So `mode::processItems` has no `setIndex` calls — those belong exclusively to `index::processItems`.

Here's the corrected TypeScript for `mode::processItems`:

```
function processItems(type: string, items?: number[]): void {
 setMode("scan");   // always first

 if (!items) return; // early exit

 switch (type) {
  case "something":
  case "others":
  case "fast":
   setMode("match");
   break;
  case "slow":
   setMode("skip");
   return;        // early return — skips loops and "complete"
  case "broken":
   setMode("error");
   break;
  default:
   setMode("idle");
 }

 // outer loop — count-- > 0
 while (count-- > 0) {
  // inner loop — i < items.length
  // (no mode mutations inside the loops)
  for (let i = 0; i < items.length; i++) {
   // items[i] < 0   → setMode("error")
   // items[i] === 0 → setMode("skip")  + continue
   // items[i] % 2 === 0 → setMode("match")
   // else         → (no mutation, falls through)
   if (items[i] < 0) {
    setMode("error");
    break; // exits inner loop per the transition to loop-merge
   } else if (items[i] === 0) {
    setMode("skip");
```

```
      continue;
    } else if (items[i] % 2 === 0) {
      setMode("match");
    }
   }
  }

  setMode("complete");
}
```

The key difference from before: `setIndex(i)` is gone entirely, and the inner loop body is now populated — because `mode::processItems` *does* have `setMode("error")`, `setMode("skip")`, and `setMode("match")` nodes with decisions (`items[i] < 0`, `items[i] === 0`, `items[i] % 2 === 0`) and loop-back transitions inside the inner loop. Those were present in the JSON all along but I had incorrectly attributed the `setIndex` to this mutator.

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	6/6	9.5/10

- After the clarification, the response mirrors the reality de-facto perfectly. For initial misunderstanding we subtract -0.5 from LAS.

Test 2 - Image

Procedure and objectives are exactly the same as with Test 1. With the difference that now we are going to be using the image of the diagram not the Json as it's representation.
Image files are added with "Add files/sources" (or by simply drag and dropping), and are obtained with using **Request Current Diagram Image** command of our implementation.

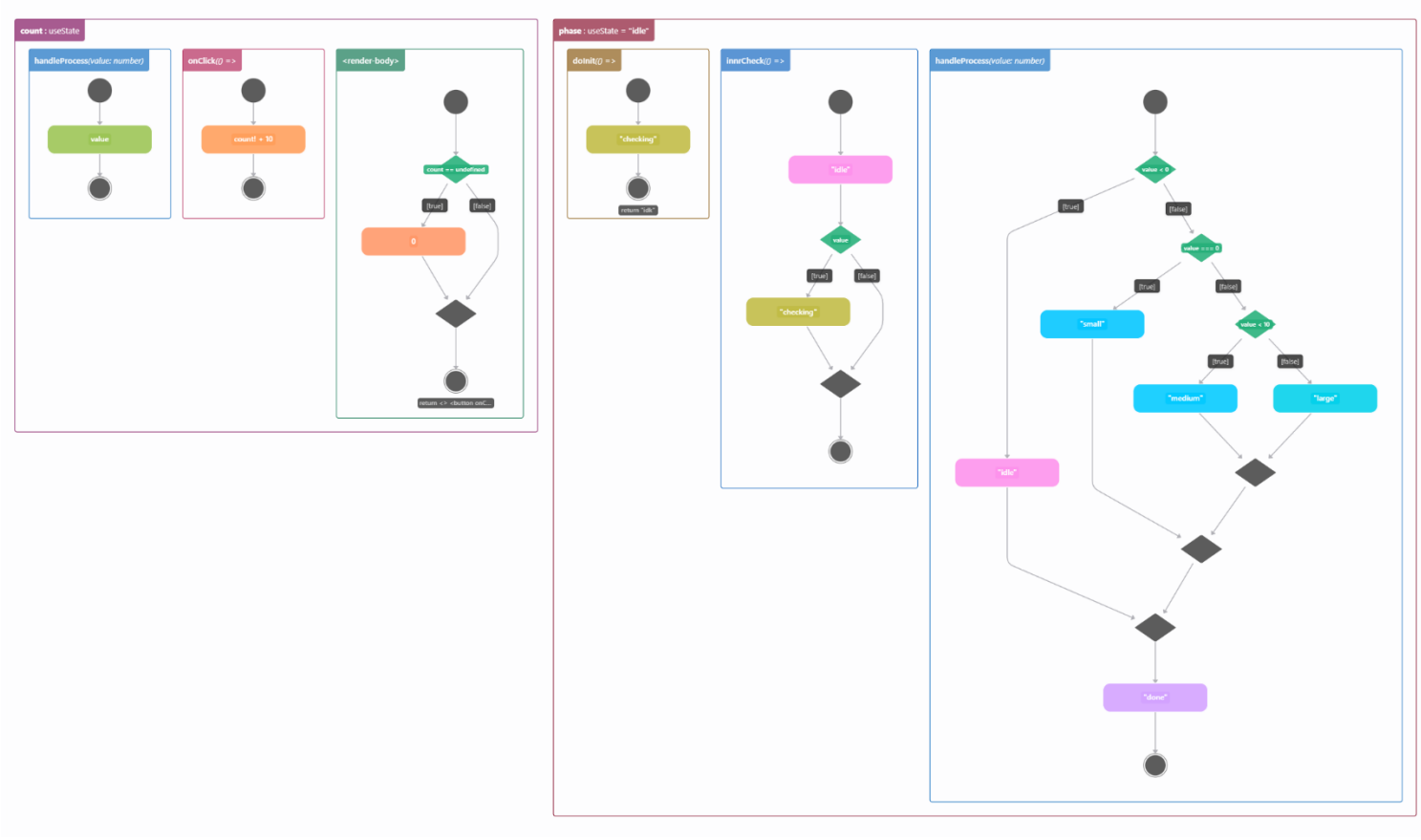
Used Prompts:

Exactly the same as in Test 1, only first prompt was slightly changed to:

I'd like to know your understanding of my diagram image representation.

Informing the model about the image (not JSON…)

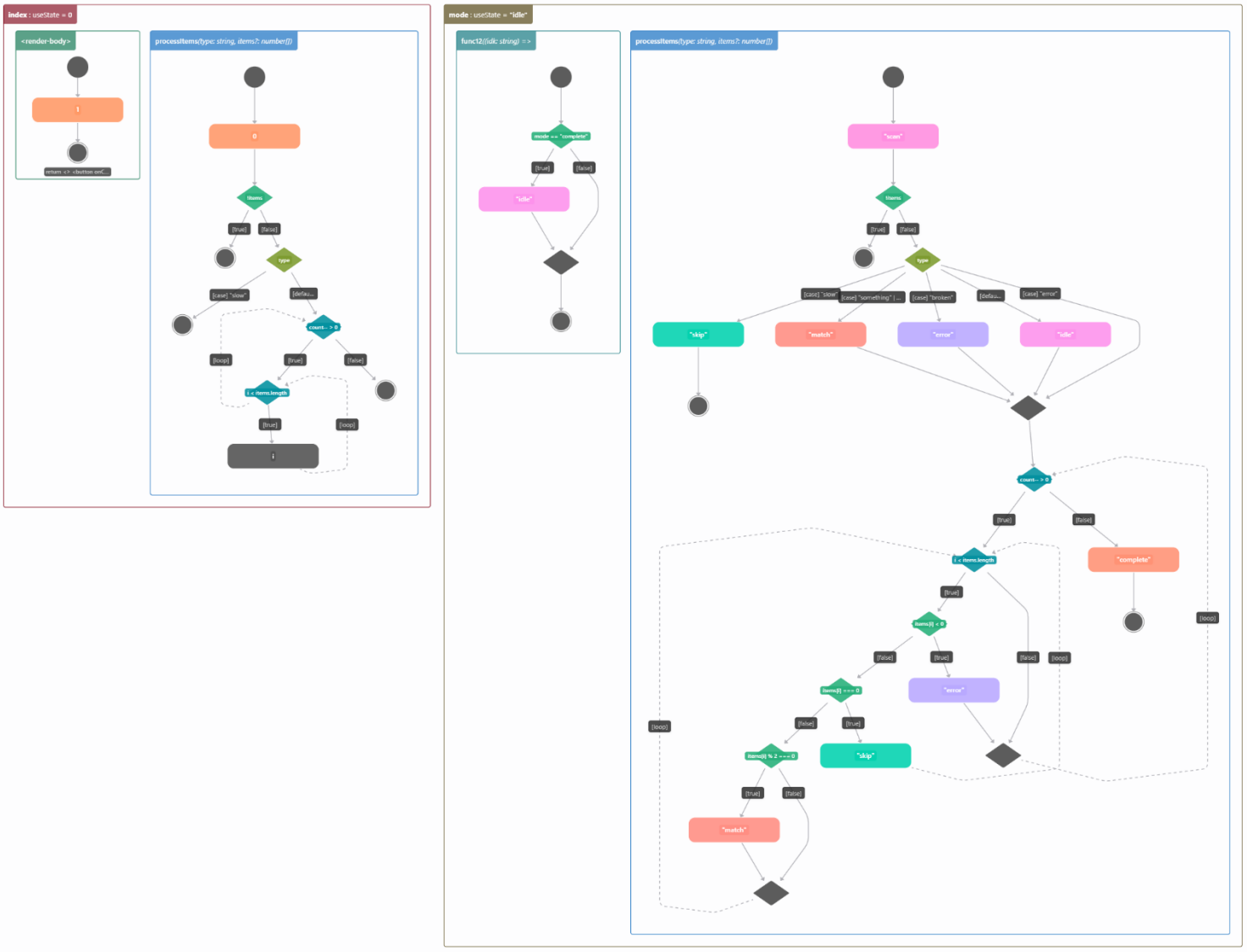
Scenario 1 - Image for ifElseChain.tsx (ifElseChain.state-diagram.png)



Resolution: 2293*1344

Estimated AI token count: 2293*1344/800 = 3852 (Note that the formula and values can differ widely across models, our formula closely mirrors the one used by Claude)

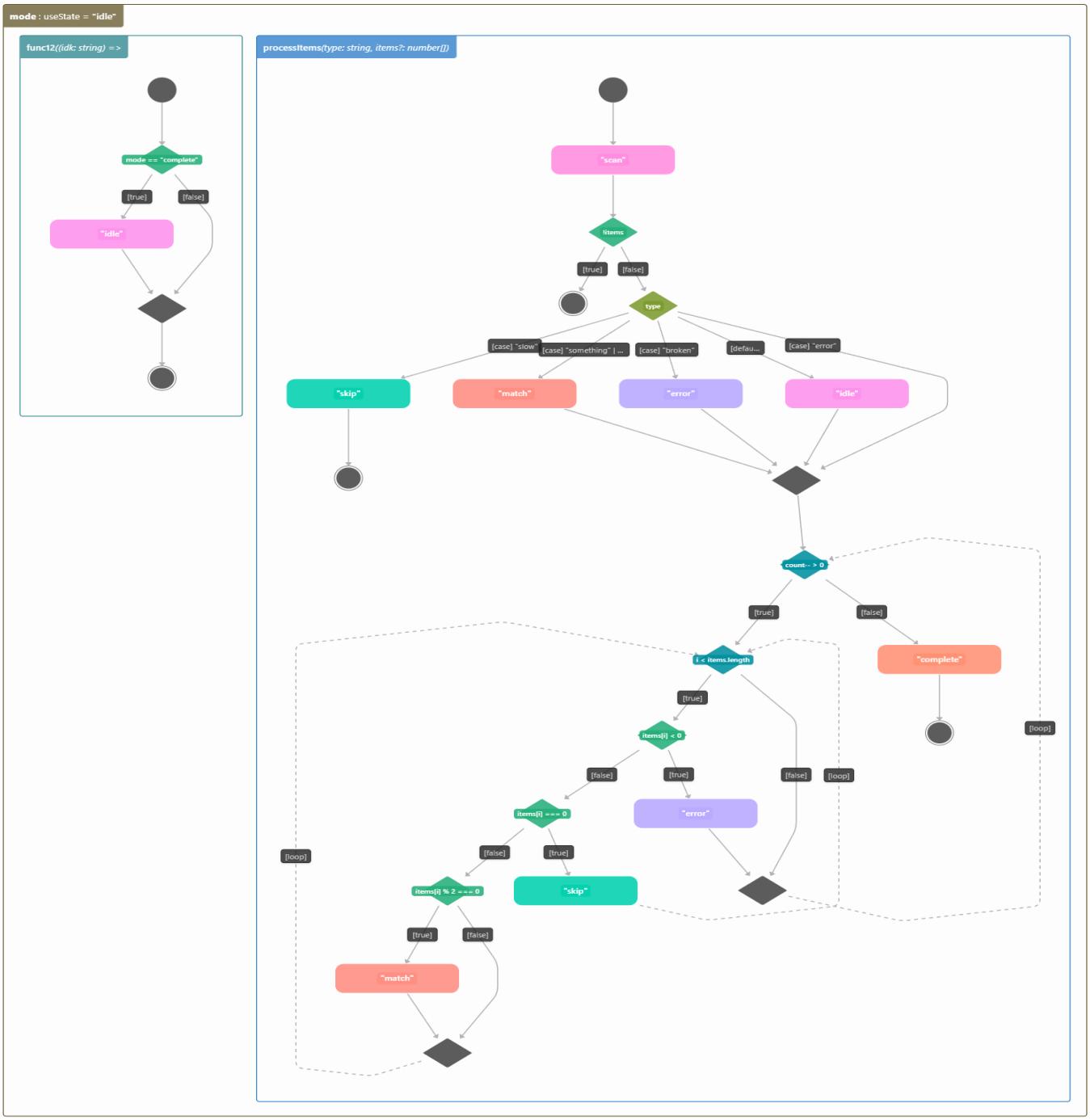
Scenario 2 - Image for switchNLoops.tsx (switchNLoops.state-diagram.png)



Resolution: 2314*1782

Estimated AI token count: 2314*1782/800 = 5154 (Note that the formula and values can differ widely across models, our formula closely mirrors the one used by Claude)

Scenario 2.5 - *Image for switchNLoops*.tsx - *processItems(…) of mode* state (switchNLoops.state-diagram-filtered.png)



Resolution: 1525*1782

Estimated AI token count: 1525*1782/800 = 3397 (Note that the formula and values can differ widely across models, our formula closely mirrors the one used by Claude)

We have decided to utilize the filter/hide feature of our implementation to provide model with image showing only the requested state variable make it significantly easier for them (this decision came after witnessing the results for the full variant).
Model will receive this image right after the Scenario 2 (unfiltered full image sample) with following prompt:

Ok, same assignment, focus on handleItems of mode state.

ChatGPT 5.3 (subscription)

Response for Scenario 1:

This diagram is essentially a **state + control-flow visualization of a React component**, combining:

- useState state management
- event handlers (onClick, handleProcess)
- conditional rendering / branching logic
- a simple state machine (idle → checking → done)

I'll walk through it in **three sections (left → right)** and explain **nodes + transitions**.

🟣 **1. Left Section — count: useState**

This part models a **counter state and how it flows into rendering**.

🔷 **Nodes**

- **Start (black circle)**
  Entry into a function.
- **handleProcess(value: number) (blue box)**
  A function that receives a number input (value).
- **value (green box)**
  Represents the incoming parameter.
- **End (double circle)**
  Function exits.

🔶 **onClick() flow**

- **Start → count + 10 → End**

This represents:

onClick={() => setCount(count + 10)}

So:

- Click triggers increment
- No branching, just a direct transformation

🔷 **<render-body>**

This is the **render logic of the component**.

Flow:

1. **Start**
2. Check: count === undefined
    - This is the **green decision diamond**
3. Branch:
    - **true → show fallback**
    - **false → show 0 (orange box)**

Then both paths:

- Merge (diamond)
- Return JSX (return <> <button onClick=...)

🧠 **Interpretation**

This section models:

- State initialization / update (count)
- UI depending on whether count exists
- Event → state update → re-render loop

🟠 **2. Middle Section — phase: useState = "idle"**

This introduces a **state machine concept**.

🔷 **doInit()**

Flow:

1. Start
2. Set "checking" (yellow box)
3. Return "idle"

So:

setPhase("checking")
return "idle"

👉 This is slightly odd logically (returns idle after setting checking), but diagram-wise:

- it shows **transition intent vs return value**

🔷 innerCheck()

Flow:

1. Start
2. Current state: "idle" (pink box)
3. Decision: value
    - if **true** → **"checking"**
    - if **false** → **skip**
4. Merge → End

🧠 Interpretation

This section models:

- A **phase state machine**
- Transitions like:
    - "idle" → "checking"
- Conditional triggering of transitions

🔵 3. Right Section — Main handleProcess(value: number)

This is the **core logic tree**.

🔷 Entry

- Start node

🔷 First decision: value < 0

- **true → "idle" (pink)**
- **false → continue**

🔷 Second decision: value === 0

- **true → "small" (blue)**
- **false → continue**

🔷 **Third decision: value < 10**

- **true → "medium"**
- **false → "large"**

🔷 **Merge flows**

All branches eventually:

1. Merge (diamond)
2. Merge again (second diamond)
3. Transition to "done" (purple)
4. End

🧠 **Interpretation**

This is a classic **if–else chain visualized as a flowchart**:

```
if (value < 0) {
  return "idle"
} else if (value === 0) {
  return "small"
} else if (value < 10) {
  return "medium"
} else {
  return "large"
}
```

Then regardless of branch:

```
setPhase("done")
```

🔁 **How Transitions Work (General Rules in Diagram)**

🟢 **Green Diamonds → Conditions**

- Represent if checks
- Outgoing edges labeled [true] / [false]

🔷 **Colored Rectangles → Actions / States**

- Blue → functions
- Green → values / expressions

- Pink → state (idle)
- Yellow → intermediate state (checking)
- Cyan → categorized outputs (small, medium, large)
- Purple → final state (done)

⚫ Circles

- Solid circle → **start**
- Double circle → **end**

🔶 Diamonds (gray)

- **Merge points**
- Multiple paths join back into one flow

🧩 Big Picture

This diagram combines **three layers of logic**:

1. UI State

- count
- affects rendering

2. Process State Machine

- phase: idle → checking → done

3. Business Logic

- classify value into:
  - negative → idle
  - 0 → small
  - <10 → medium
  - ≥10 → large

⚠️ Subtle Observations (worth noting)

- "idle" is used both:
  - as a **state**
  - as a **result of negative values**
- "checking" appears as a **transient state**

- Final convergence always leads to "done" → suggests a **workflow completion**

✅ **Short Summary**

- **Left**: state + UI rendering (count)
- **Middle**: phase transitions (idle ↔ checking)
- **Right**: main decision tree → categorizes input → ends in "done"

If you want, I can:

- rewrite this as actual React code
- simplify the diagram
- or point out design issues / improvements (there are a few)

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
2/2	6/6	8.5/9	6.5/10

- -0.5 subtracted from US for a minor misinterpreting of `count! + 10` as `count + 10` (missing '!').
- -2 from LAS for swapping true and false branches logic in <render-body>
- -0.5 from LAS for saying that *solnit* of *phase* return "idle" instead of "idk", but this is not major.
- -1 from LAS for stating that *handleProcess* of *phase* returns "idle", "small", "medium", and "large" conditionally and only sets phase to "done". Diagram clearly does not show any such a returns, overall reasoning was however correct.

Response for Scenario 2:

Alright—this one is doing a lot more. I'll stay disciplined and focus on **processItems(type: string, items: number[]) inside the mode state section**, then give you a clean structural summary at the end.

🔵 **Core Focus: processItems(...) (mode state)**

This is essentially a **hybrid of:**

- a **switch-case on type**
- a **loop over items**
- nested **value classification logic**
- and **state emissions (scan, match, error, skip, complete)**

🟣 **1. Entry + Initial State**

Flow:

1. **Start node**
2. Emit "scan" (pink)

👉 This suggests:

setMode("scan")

This is the **initial processing phase**.

🟢 **2. First Gate — items**

Decision:

- **If no items → End immediately**
- **If items exist → continue**

So:

if (!items) return

🟡 **3. Switch on type**

This is the first major branching structure.

Cases:

🟢 **"slow" → "skip"**

- Direct exit path

return "skip"

🔴 **"something" → "match"**

- Goes to "match" state
- Then merges with other flows

🟣 **"broken" → "error"**

- Emits "error"
- Continues to merge

🟣 **"error" → "idle"**

- Explicit fallback to "idle"

⚙️ **default → merge**

- Falls through into general processing

🧠 **Interpretation**

This is a **pre-processing classification** based on type:

- Some cases **short-circuit**
- Others **set an intermediate state and continue**

🔷 **4. Merge → Loop Entry Decision**

After the switch:

Decision: count >= 0

- **false → "complete" → END**
- **true → enter loop**

👉 This is your **loop guard**

🔁 5. Main Loop Block

This is the most complex part.

Loop condition:

i < items.length

Classic iteration:

for (let i = 0; i < items.length; i++)

🧩 6. Per-Item Logic

Inside the loop:

🔷 Step 1: items[i] < 0

- **true → "error"**
- **false → continue**

🔷 Step 2: items[i] === 0

- **true → "skip"**
- **false → continue**

🔷 Step 3: items[i] >= 0

- **true → "match"**
- **false → loop back**

🧠 Interpretation

Each item is classified:

```
if (items[i] < 0) {
  state = "error"
} else if (items[i] === 0) {
```

```
  state = "skip"
} else {
  state = "match"
}
```

🔁 **7. Loop Behavior**

- After each classification:
  - flows merge
  - loop continues (i++)
- Eventually exits when:

```
i >= items.length
```

🟠 **8. Completion Path**

Once loop exits:

1. Merge
2. "complete" (orange)
3. End

👉 Final state:

```
setMode("complete")
```

🧠 **Behavioral Summary (processItems)**

This function behaves like:

Phase 1: Setup

- Set "scan"
- Validate items

Phase 2: Type-based routing

- Some types:
  - **exit early** (skip)
  - **set immediate state** (match, error, idle)
- Others continue

Phase 3: Iterative evaluation

- Loop over all items
- Classify each item into:
    - "error" (< 0)
    - "skip" (= 0)
    - "match" (> 0)

Phase 4: Finalization

- When loop completes → "complete"

⚠️ **Important Observations**

1. Mixed responsibilities

This function is doing:

- control flow
- classification
- state mutation
- iteration

👉 It's effectively a **mini interpreter**

---

2. State overwriting

Inside loop:

- "error", "skip", "match" overwrite each other
- No accumulation → **last item wins**

---

3. Early exits vs continuation

- "slow" exits immediately
- "broken" / "something" continue

👉 inconsistent control semantics

---

4. Redundant condition

- items[i] >= 0 after already checking < 0 and === 0

👉 logically unnecessary branch

📊 **Final Summary (as requested)**

🔢 **State Variables**

1. mode

- values observed:
  - "idle"
  - "scan"
  - "match"
  - "error"
  - "skip"
  - "complete"

2. index (from left section, implied loop counter)

⚙️ **Mutators (State Setters)**

Explicit / implied:

- setMode(...)
  - used in:
    - "scan"
    - "match"
    - "error"
    - "skip"
    - "complete"
    - "idle"
- setIndex(...) (implied via loop i)

📌 **Counts**

- **State variables:** 2
  - mode
  - index
- **Mutators:** 2
  - setMode
  - setIndex (implicit loop control)

✅ **One-line Mental Model**

A state-driven processor that scans items, classifies them per type and value, iterates through all entries, and

finishes in a "complete" state—unless short-circuited early.

If you want next step, I can:

- flatten this into clean code
- or show how to refactor into a proper state machine (this would benefit a lot)

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	6/6	3.5/10

- -2 from LAS for incorrectly swapping state sets for default and “error” case in switch.
- -2.5 from LAS for misreading first loops condition as `count >= 0` instead of `count-- > 0` and for further misclassifying it as a decision instead of loop. Later failing to acknowledge that there is a path that loops back to it (we decided to count this as a 1 major and one minor misread mistake).
- -2 from LAS for incorrectly stating that third decision is `items[i] >= 0` instead of `items[i] % 2 === 0` (deemed sever enough and too behavior changing for a simple misread)

Response for Scenario 2.5:

This diagram focuses on **`mode: useState = "idle"`**, especially:
`processItems(type: string, items?: number[])`

`processItems(...)` flow

1. Start → **`"scan"`**

The function begins by transitioning `mode` to:
`"scan"`

So the process has entered a scanning phase.

---

2. Check **`items`**

Decision node:
`items`

- **true** → exits early
- **false** → continues to `type`

This suggests the code may be checking something like:
`if (items) return`

or the diagram's true/false labels may be inverted from the source logic. Based on the later loop over `items.length`, I would expect the intended logic to be:
`if (!items) return`

So this is one possible ambiguity.

---

3. Switch / branch on **`type`**

The `type` node branches into several cases:

`case "slow"` → **`"skip"`** → **end**

This is an early terminal branch.

`case "something" ...` → **`"match"`**

This sets mode to `"match"` and then rejoins the main flow.

`case "broken"` → **`"error"`**

This sets mode to `"error"` and rejoins.

`case "error"` → `"idle"`

This resets mode back to `"idle"` and rejoins.

`default`

This skips directly into the shared continuation path.

---

4. Shared continuation → `count-- > 0`

After the switch branches merge, the next decision is:
`count-- > 0`

- **true** → enters the item-processing loop
- **false** → `"complete"` → end

So this acts like an outer loop / retry budget / countdown guard.

---

5. Main item loop

Inside the dashed loop region, the diagram checks:
`i < items.length`

- **true** → inspect `items[i]`
- **false** → loop exits toward the outer loop continuation

---

6. Per-item classification

For each item:

Check 1
`items[i] < 0`

- **true** → `"error"`
- **false** → continue

Check 2
`items[i] === 0`

- **true** → `"skip"`
- **false** → continue

Check 3
`items[i] % 2 === 0`

- **true** → `"match"`
- **false** → no state change, then merge

So item handling looks like:

```
if (items[i] < 0) {
  setMode("error")
} else if (items[i] === 0) {
  setMode("skip")
} else if (items[i] % 2 === 0) {
  setMode("match")
}
```

Odd positive numbers appear to pass through without emitting a new mode.

Overall behavior

`processItems`:

1. sets mode to `"scan"`
2. checks whether `items` exists
3. branches on `type`
4. may emit `"skip"`, `"match"`, `"error"`, or `"idle"`
5. runs a countdown-controlled loop
6. iterates through `items`
7. classifies each item
8. eventually emits `"complete"`

Summary

- **State variables shown:** 1
  - `mode`
- **Observed mode values:** 6
  - `"idle"`, `"scan"`, `"skip"`, `"match"`, `"error"`, `"complete"`
- **Mutator implied:** 1
  - `setMode(...)`
- **Major constructs:** switch-case, outer countdown loop, inner array loop, nested if/else chain.

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	6/6	5.5/10

- We can observe that he is not sure about `!items` or` items`, after hesitation and additional reasoning settling at !items which is correct. -0.5 from LAS for initially stating `items`.
- -2 from swapping default and idle case in switch (just like before).
- -2 from failing to decisively mention the first outer loop as well as failing to acknowledge the path looping back to the outer loop, from “skip” (just like before).

Claude Sonnet 4.6

Response for Scenario 1:

This is a **React component state diagram** showing the flow of a component that uses two `useState` hooks and several functions. Let me walk through each section:

Left Panel — `count : useState`

This panel has three sub-diagrams:

`handleProcess(value: number)`

- Takes a number value as input
- Simply returns/sets that **`value`** — a straightforward pass-through action.

`onClick() =>`

- On click, computes **`count + 10`** and updates the count state.

`<render-body>`

- On render, checks: **`count == undefined`**
  - **[true]** → renders `0` (default fallback), then hits a merge diamond, and returns a `<button onClick=...>` JSX element.
  - **[false]** → skips the `0` node, goes straight to the merge diamond, same return.

Right Panel — `phase : useState = "idle"`

This panel tracks a `phase` state, initialized to `"idle"`, with three functions:

`doInit() =>`

- Sets phase to **`"checking"`**
- Then immediately returns **`"idle"`** (resets or signals idle after init).

`innrCheck() =>`

- Starts at the initial node, then checks the **`value`** condition:
  - **[true]** → sets phase to **`"checking"`**
  - **[false]** → skips it
- Both paths converge at a diamond (merge), then terminate.

`handleProcess(value: number)` ***(the main logic)***

This is the most complex flow — an **if/else chain** (hence the diagram name `ifElseChain`):

1. **`value < 0`?**

- **[true]** → sets phase to `"idle"` (pink), then flows down to merge
- **[false]** → continues to next check ↓

2. **`value === 0`?**
   - **[true]** → sets phase to `"small"` (cyan)
   - **[false]** → continues to next check ↓
3. **`value < 10`?**
   - **[true]** → sets phase to `"medium"` (cyan)
   - **[false]** → sets phase to `"large"` (cyan)
4. All branches converge through a series of **merge diamonds**, finally setting phase to `"done"` (purple), then reaching the **terminal state**.

Summary of the Logic

The diagram models a React component that:

- Tracks a **`count`** (number) and a **`phase`** (string) via two `useState` hooks
- On button click, increments count by 10
- `handleProcess` categorizes a numeric value into phases: `"idle"` (negative), `"small"` (zero), `"medium"` (1–9), `"large"` (10+), ending in `"done"`
- The `if/else chain` in `handleProcess` is the core decision tree of the component

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
2/2	6/6	8.5/9	7.5/10

- -0.5 from US for misinterpreting of `count! + 10` as `count + 10` (missing '!', similar to ChatGPT).
- -0.5 from LAS for misreading return of doInit as "idle" instead of "idk" (again same as ChatGPT).
- -2 for overstating that *innrCheck* simply starts at initial node and imidiatly going to the decision, completely omitting the "idle" state in between which changes logic completely.

Response for Scenario 2:

`processItems(type: string, items?: number[])` — from `mode : useState = "idle"`

This is the complex one. It's a **switch + nested loop** logic flow.

Step 1 — Switch on `type`

First, it sets mode to `"case"` (pink), then checks if `items` exists:

- **[false]** → terminal (early exit, no items to process)
- **[true]** → proceeds to a **`type` switch diamond** with four branches:

Case	Sets mode to
`"slow"`	**`"skip"`** (teal) → terminal
`"something"`	**`"match"`** (orange-red)
`"broken"`	**`"error"`** (purple)
`[default]` / `"error"`	**`"idle"`** (pink)

After `"match"`, `"error"`, and `"idle"` resolve, they **converge** at a merge diamond and continue downward.

Step 2 — `count > 0` check

- **[false]** → sets mode to **`"complete"`** (orange) → terminal
- **[true]** → enters the **loop** (dashed border region)

Step 3 — Loop: `i < items.length`

This is a `for` loop iterating over the `items` array:

[true] — loop body:

1. **`items[i] < 0`?**
    - **[false]** → check `items[i] === 0`
        - **[false]** → check `items[i] % 2 === 0`
            - **[true]** → sets mode to **`"match"`**
            - **[false]** → (falls through)
        - **[true]** → sets mode to **`"skip"`** (teal)
    - **[true]** → sets mode to **`"error"`** (purple)
2. All inner branches converge → loop back **[loop]** to `i < items.length`

[false] — loop exits → final merge diamond → terminal

Pseudocode Summary

```
function processItems(type, items) {
 setMode("case");
 if (!items) return;

 switch (type) {
  case "slow":    setMode("skip");  return;
  case "something": setMode("match"); break;
  case "broken":  setMode("error"); break;
  default:        setMode("idle");  break;
 }

 if (count <= 0) { setMode("complete"); return; }

 for (let i = 0; i < items.length; i++) {
  if (items[i] < 0)      setMode("error");
  else if (items[i] === 0) setMode("skip");
  else if (items[i] % 2 === 0) setMode("match");
 }
}
```

Overall Diagram Summary

	index : useState	mode : useState
Initial value	`0`	`"idle"`
Mutator functions	`processItems(type, items?)`	`func12(idk), processItems(type, items?)`
Render	`<render-body>` → returns `<button>`	—

Metric	Count
State variables	2 (`index`, `mode`)
Mutator functions	3 total (`processItems` ×2 scoped separately, `func12`)
Unique function signatures	2 (`func12`, `processItems`)
Possible mode values	6 (`"idle"`, `"case"`, `"skip"`, `"match"`, `"error"`, `"complete"`)

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	5.5/6	5.5/10

- -0.5 from US for misreading “scan” state as “case”
- -0.5 from LAS for inverting the `!items` to `items` and swapping true and false transitions, this is semantically equivalent but a minor misinterpretation nevertheless.
- -2 from LAS for merging the default case with “error” for the switch.
- -2 from LAS for misclassifying the first loop as a decision and later failing to acknowledge that there is a path that loops back to it (similar to ChatGPT)

Response for Scenario 2.5:

Thanks for the clearer image! I can now see some corrections from my previous reading:

`processItems(type: string, items?: number[]) ── mode : useState = "idle"`

Step 1 — Sets mode to `"scan"`

First action is always setting mode to **`"scan"`** (pink), then checks if **`items`** exists:

- **[false]** → goes to the **`type`** **switch** directly
- **[true]** → hits a **terminal** (early exit)

⚠ Correction from before: `"scan"` not `"case"`, and the `items` truthy/falsy branches are **swapped** — `[false]` continues, `[true]` exits early.

Step 2 — Switch on `type`

Four branches off the `type` diamond:

Case	Sets mode to	Then
`"slow"`	**`"skip"`** (teal)	→ terminal (full exit)
`"something"`	**`"match"`** (salmon)	→ merge
`"broken"`	**`"error"`** (purple)	→ merge
`[default]` / `"error"`	**`"idle"`** (pink)	→ merge

`"match"`, `"error"`, `"idle"` all converge at a merge diamond and continue.

Step 3 — `count > 0` check

- **[false]** → sets mode to **`"complete"`** (orange) → terminal
- **[true]** → enters the **loop**

Step 4 — Loop: `i < items.length`

[false] on loop condition → exits loop → **[loop]** back up to `count > 0` check (dashed arrow)

[true] — loop body, three nested checks:

1. **`items[i] < 0`?**
   - **[true]** → sets mode to **`"error"`** → merge → **[loop]** back
   - **[false]** → next check ↓

2. `items[i] === 0`?
    - **[true]** → sets mode to `"skip"` (teal) → merge → **[loop]** back
    - **[false]** → next check ↓
3. `items[i] % 2 === 0`?
    - **[true]** → sets mode to `"match"` (salmon)
    - **[false]** → falls through
    - Both → merge diamond → **[loop]** back to `i < items.length`

Corrected Pseudocode

```
function processItems(type, items) {
 setMode("scan");
 if (items) return; // exits if items is truthy

 switch (type) {
  case "slow":     setMode("skip");  return;
  case "something": setMode("match"); break;
  case "broken":   setMode("error"); break;
  default:        setMode("idle");  break;
 }

 if (count <= 0) { setMode("complete"); return; }

 for (let i = 0; i < items.length; i++) {
  if     (items[i] < 0)       setMode("error");
  else if (items[i] === 0)     setMode("skip");
  else if (items[i] % 2 === 0)  setMode("match");
 }
 // loop end feeds back to count > 0 check
}
```

`func12(idc: string) =>`

Simple guard function:

- Checks `mode === "complete"`
    - **[true]** → sets mode to `"idle"`
    - **[false]** → skips
- Both paths merge → terminal

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	6/6	5.5/10

He manages to realize his mistake with “scan” -> “case” state misread which he receives +0.5 US in this case, but after that proceeds to do exactly the same mistakes as before.

Test 3 - JSON + Image

Our objectives, approaches and metrics stay unchanged. This time we will combine both JSON and Image in a single prompt.
Both JSON as well as Image representation data will mirror the ones used in Test 1 and Test 2 one to one.

Used prompts:

Used prompt will be exactly the same with the exception the first one:

I'd like to know your understanding of my diagram representations.

This time simply stating that there will be multiple representations of it…

ChatGPT 5.3 (subscription)

Response for Scenario 1:

This is a really solid diagram—it's essentially a **control-flow + state-transition hybrid** for a React component. I'll walk through how I interpret it so you can see if it matches your intent.

🧠 Big Picture (What this diagram represents)

You're modeling:

- **React state (`useState`)**
- **Functions as flow graphs**
- **State updates as nodes**
- **Control flow (if/else) explicitly as decision + merge nodes**

So this is not just a flowchart—it's closer to a **static control-flow graph (CFG)** enriched with **state mutations**.

🧩 Core Node Types (Your visual language)

From the diagram, I interpret:

1. Entry / Exit

- ⚫ **Filled circle (top)** → function entry
- ⚫ **Double circle (bottom)** → function exit

---

2. State Updates

- 🟩🟨🟦 colored rounded rectangles → **state assignments**
  - Example: `"idle"`, `"checking"`, `"small"`
- Color appears to encode **which state variable**:

- `phase` → pink/yellow/blue
- `count` → green/orange

---

3. Decisions

- 🔷 Diamond → **boolean condition**
  - e.g. `value < 0`, `value === 0`
- Outgoing edges:
  - `[true]`
  - `[false]`

---

4. Merge Nodes

- 🔶 Diamond (lower) → **merge/join point**
  - reconverges branches after conditionals

---

5. Transitions (Edges)

- Arrows = execution flow
- Labeled edges = conditional branches
- Unlabeled = sequential flow

📦 Section 1: `count : useState`

🔷 **`handleProcess(value: number)`**

```
entry → setCount(value) → exit
```

Very straightforward:

- Takes input
- Writes it into `count`

🔷 **`onClick`**

```
entry → setCount(count! + 10) → exit
```

So:

- Uses current `count`
- Adds 10

🔷 `<render-body>`

Flow:

```
entry
  ↓
[count == undefined?]
   ├─ true → setCount(0)
   └─ false → (skip)
        ↓
     merge
        ↓
     return JSX
```

Interpretation:

- Lazy initialization safeguard
- Ensures `count` is never undefined before render completes

 Section 2: `phase : useState`

Initial:

```
phase = "idle"
```

🔷 `doInit()`

```
entry → setPhase("checking") → exit ("idk")
```

Simple:

- Always sets `"checking"`
- Returns `"idk"` (not used in flow)

🔷 `innrCheck(value)`

Flow:

```
entry
  ↓
setPhase("idle")
  ↓
[value ?]
   ├─ true → setPhase("checking")
   └─ false → (skip)
        ↓
     merge
        ↓
      exit
```

Meaning:

- Always resets to `"idle"`
- Conditionally moves to `"checking"`

🔥 Section 3: `handleProcess(value: number)` (Main Logic)

This is the most complex—and your diagram handles it nicely.

Full Flow (flattened)

```
entry
  ↓
[value < 0?]
  ├─ true → setPhase("idle")
  └─ false →
       ↓
    [value === 0?]
       ├─ true → setPhase("small")
       └─ false →
            ↓
         [value < 10?]
            ├─ true → setPhase("medium")
            └─ false → setPhase("large")
```

Then everything merges:

```
(all branches)
   ↓
merge → merge → merge
   ↓
setPhase("done")
   ↓
exit
```

🧠 Key Insight

This is a **classic if / else-if / else chain**, represented as:

- Nested decisions
- Followed by **structured merges**
- Then a **single final action**

💡 Semantically Equivalent Code

```
if (value < 0) {
  setPhase("idle");
} else if (value === 0) {
  setPhase("small");
} else if (value < 10) {
  setPhase("medium");
} else {
```

```
  setPhase("large");
}

setPhase("done");
```

🔄 Transitions Explained

Types of transitions you use:

1. **Sequential**
   `A → B`

   Always happens

2. **Conditional**
   `decision → branch [true/false]`

3. **Converging**
   `multiple → merge node`

4. **Post-merge continuation**
   `merge → next step`

🧠 What You're Really Modeling (Important)

This diagram encodes **three layers at once**:

1. Control Flow (CFG)

- Decisions
- Branching
- Merging

2. State Transitions

- `setPhase(...)`
- `setCount(...)`

3. Execution Order Guarantees

- Especially the fact that:
  `setPhase("done")`
  `ALWAYS runs`

⚠️ Subtle but Important Behavior

Your diagram makes something very clear that code often hides:

👉 Multiple `setPhase(...)` calls happen in sequence
👉 Final state is always `"done"`

So intermediate states like `"small"`, `"medium"` etc. are **transient**

👍 What You Did Well

- Clean separation by **state variable**
- Explicit **merge nodes** (most diagrams skip this—yours is more correct)
- Proper modeling of **if-else chains**
- Clear **entry/exit semantics**
- Color encoding for state domains

👊 Minor Ambiguities / Suggestions

1. Decision Labels

- `value` (in `innrCheck`) is vague
  → could be `Boolean(value)` or `value !== 0`

---

2. Merge Node Semantics

- You use identical diamonds for:
  - decision
  - merge

Consider:

- Decision → 🔷
- Merge → ◇ (or labeled)

---

3. State vs Action

Right now:

- `"small"` looks like a state *value*

But visually it could be mistaken for:

- UI label

- output

Maybe annotate:
`setPhase("small")`

📜 Final Interpretation (Concise)

Your diagram is:

A **control-flow graph of a React component**, where:

- nodes represent **state mutations or control operations**
- edges represent **execution paths**
- diamonds encode **branching and merging**
- layout groups behavior by **state variable and function**

If you want, I can:

- formalize this into a **diagram spec / grammar**
- or help you turn this into a **tool (AST → graph renderer)**
- or critique it from a **compiler / static analysis perspective**

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
2/2	6/6	9/9	10/10

Response for Scenario 2:

Yes — focusing only on **`processItems(type, items?)` for `mode`**.

This graph says:

`processItems(type: string, items?: number[])`

starts by setting:

`setMode("scan")`

Then it checks whether `items` exists.

If `!items` is true:

`return`

So the only mode update is:

`"scan"`

If `items` exists, execution continues into the `switch(type)`.

For `type`:

`"something" | "others" | "fast"`

it sets:

`setMode("match")`

For:

`"slow"`

it sets:

```
setMode("skip")
return
```

So `"slow"` exits early and never reaches `"complete"`.

For:

`"broken"`

it sets:

`setMode("error")`

For:

`"error"`

there is a case path, but no state update before the merge.

For any default case:

`setMode("idle")`

All non-early-return switch paths then merge and enter the loop section.

The loop structure is nested:

```
while (count-- > 0) {
  while (i < items.length) {
    ...
  }
}
```

Inside the inner loop, each item is classified:

```
if (items[i] < 0) {
  setMode("error")
}
```

A negative item sets `"error"` and exits the inner-loop body path into the outer loop merge.

Otherwise:

```
else if (items[i] === 0) {
  setMode("skip")
}
```

Zero items set `"skip"` and loop back.

Otherwise:

```
else if (items[i] % 2 === 0) {
  setMode("match")
}
```

Positive even items set `"match"` and loop back.

If none of those are true, the item causes no `mode` update and loops back.

When the inner loop finishes, control returns to the outer loop condition. When the outer loop finishes:

```
setMode("complete")
```

Then the function exits.

So the important behavior is:

```
"complete" is the final mode for every path that reaches the end.
```

But not every path reaches the end:

```
!items        → final visible update is "scan"
type=="slow" → final visible update is "skip"
otherwise    → final update is "complete"
```

Summary:

Item	Count
State variables	2

Total mutators	4
`mode` mutators	2
`index` mutators	2
`mode` updates inside `processItems`	9
Early exits in `mode.processItems`	2

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	6/6	10/10

Claude Sonnet 4.6

Response for Scenario 1:

Here's a thorough breakdown of the diagram, which represents a React component called **IfElseChainComplex** with two state variables and several functions.

Two State Containers (the two big bordered boxes)

Left box — `count : useState`

Tracks a numeric counter. Three functions mutate it:

- **`handleProcess(value: number)`** — sets `count` to `value` directly.
- **`onClick(() =>)`** — increments `count` by 10 (`count! + 10`).
- **`<render-body>`** — the component's render logic, which also initializes `count`.

Right box — `phase : useState = "idle"`

Tracks a workflow phase string. Initial value is `"idle"`. Possible states: `"idle"` → `"checking"` → `"small"` / `"medium"` / `"large"` → `"done"`. Three functions mutate it.

Functions & Their Flow Diagrams

`handleProcess(value: number)` ***(left box, blue)***

Simple passthrough — sets `count = value`, then exits. No branching.

`onClick(() =>)` ***(left box, orange)***

Simple passthrough — sets `count = count! + 10`, then exits. No branching.

`<render-body>` ***(left box, teal)***

The component's render function:

1. Checks `count == undefined`
   - **[true]** → sets `count = 0` (initialization guard)
   - **[false]** → skips
2. Both paths merge → renders the button JSX (`<button onClick={...}>Run</button>` etc.) and exits.

`doInit(() =>)` ***(right box, olive)***

1. Enters → sets `phase = "checking"`
2. Returns/exits with `"idk"` label (early return marker)

Straightforward — always sets phase to checking, no branching.

`innrCheck(() =>)` ***(right box, pink/magenta)***

1. Enters → immediately sets `phase = "idle"`
2. Hits decision on **`value`**:
    - **[true]** → sets `phase = "checking"` again
    - **[false]** → skips
3. Both paths merge → exits

So `innrCheck` resets to idle first, then conditionally bumps back to checking if `value` is truthy.

`handleProcess(value: number)` ***(right box, cyan — the main logic)***

This is the most complex function — a chained if/else tree classifying the number:

```
value < 0?
├── [true]  → phase = "idle"
└── [false] → value === 0?
        ├── [true]  → phase = "small"
        └── [false] → value < 10?
                ├── [true]  → phase = "medium"
                └── [false] → phase = "large"
```

All four branches merge → then merge again → `phase = "done"` → exits.

So **every path through `handleProcess` ends with `phase = "done"`**, regardless of which intermediate phase was set.

Visual Convention Legend

Shape	Meaning
Filled circle (top)	Entry point
Filled circle (bottom, double ring)	Exit point
Rounded rectangle (colored)	State update (`setPhase` or `setCount`)
Diamond	Decision / branch point
Plain diamond (no label)	Merge point (paths converging)
Arrows with `[true]`/`[false]`	Conditional branch directions

Summary

The diagram is essentially a **state-machine visualizer for a React component**, showing how each function transitions `phase` and `count` through state updates, with branching logic rendered as flowchart diamonds and merge nodes tracking where conditional paths rejoin.

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
2/2	6/6	9/9	10/10

Response for Scenario 2:

Here's the focused breakdown of **`processItems`** as a mutator of the `mode` state, followed by a summary.

`processItems(type: string, items?: number[])` — `mode` state transitions

Phase 1 — Entry & Guard

1. Sets `mode = "scan"`
2. Checks `!items`:
   - **[true]** → exits immediately (early return, no further state changes)
   - **[false]** → continues to the switch

Phase 2 — Switch on `type`

Case	`mode` set to	Then...
`"slow"`	`"skip"`	**exits immediately** (early return)
`"something"` | `"others"` | `"fast"`	`"match"`	falls through to loop
`"broken"`	`"error"`	falls through to loop
`"error"`	*(no change)*	falls through to loop
`[default]`	`"idle"`	falls through to loop

Phase 3 — Outer loop: `count-- > 0`

- **[false]** (count exhausted) → sets `mode = "complete"` → exits
- **[true]** → enters inner loop

Phase 4 — Inner loop: `i < items.length`

- **[false]** → loops back to outer `count-- > 0` check
- **[true]** → enters per-item decision chain:

```
items[i] < 0?
├── [true]  → mode = "error"  → exits inner loop entirely
└── [false] → items[i] === 0?
        ├── [true]  → mode = "skip"  → [loop] back to i < items.length
        └── [false] → items[i] % 2 === 0?
                ├── [true]  → mode = "match" → merge → [loop] back
                └── [false] → merge → [loop] back
```

So per item: negative → error + break inner; zero → skip + continue; even → match + continue; odd → no change + continue.

After the inner loop exhausts all items, it loops back to the outer `count--` check. Once count hits 0,

```
mode = "complete".
```

Component Summary

	Count
State variables	2 (`mode`, `index`)
Mutators of `mode`	2 (`func12`, `processItems`)
Mutators of `index`	2 (`<render-body>`, `processItems`)
Total unique mutator functions	3 (`func12`, `processItems`, `<render-body>`)
Possible `mode` values	6 (`"idle"`, `"scan"`, `"match"`, `"skip"`, `"error"`, `"complete"`)
Loops in `processItems`	2 (nested: outer `count-- > 0`, inner `i < items.length`)

Evaluated results:

State variables identified	Mutator functions identified	Unique states identified	Logic analyses score
1/1	1/1	6/6	10/10